

FIAP  ONI

06

IA GENERATIVA: CRIATIVIDADE CODIFICADA

LISTA DE FIGURAS

Figura 1 - Deep Learning é uma subárea de Machine Learning que, por sua vez, é uma subárea da Inteligência Artificial.....	5
Figura 2 - Acima é mostrado um modelo discriminativo. Abaixo, modelo generativo.....	7
Figura 3 - Modelos generativos de geração de áudios.....	9
Figura 4 - Modelos generativos de geração de vídeo	10
Figura 5 - Vibe Coding: humanos desenvolvendo códigos com auxílio de IA	11
Figura 6 - Alphafold: Modelo generativo para gerar estruturas 3D de proteínas ainda desconhecidas.....	12
Figura 7 - ELIZA.....	13
Figura 8 - Cadeias de Markov	14
Figura 9 - Long Short-Term Memory Networks predizendo a próxima palavra com base nas palavras anteriores	15
Figura 10 - Variational Autoencoders	16
Figura 11 - GANs	16
Figura 12 - Arquitetura Transformer e Mecanismo de Atenção.....	17
Figura 13 - Modelo de geração de imagens da OpenAI.....	18
Figura 14 - Modelo Multimodais	19
Figura 15 - Consumo de energia de modelos de IA Generativa.....	29
Figura 16 - Alucinações e a dificuldade de se dar a instrução correta para a resposta desejada.....	30

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Vetores no NumPy.....	33
Código-fonte 2 - Matrizes no NumPy	34
Código-fonte 3 - Multiplicação de matrizes	36
Código-fonte 4 - Invertendo uma matriz	37
Código-fonte 5 - Determinante de uma matriz.....	38
Código-fonte 6 - Norma de uma matriz	39
Código-fonte 7 - Distâncias entre vetores	40
Código-fonte 8 - Matriz transposta e matriz identidade	41
Código-fonte 9 - Função Linspace	41
Código-fonte 10 - Funções zeros e ones	41
Código-fonte 11 - Concatenando matrizes e vetores	42
Código-fonte 12 - Função Reshape	42

SUMÁRIO

1 IA GENERATIVA: CRIATIVIDADE CODIFICADA	5
1.1 O que é um Modelo Generativo?	6
1.2 Aplicações de Modelos Generativos	7
1.3 História da IA Generativa	12
1.4 Foundation Models e Ecossistema.....	20
1.4.1 O advento dos Transformers.....	20
1.4.2 Foundation Models.....	21
1.5 Implicações Éticas, Legais e Sociais.....	24
1.6 Desafios e Limitações	28
2 LABORATÓRIO PYTHON: FUNDAMENTOS DE ÁLGEBRA LINEAR PARA IA ...	32
2.1 Vetores	32
2.2 Matrizes.....	34
2.2.1 Multiplicação de matrizes	35
2.2.2 Outras operações relevantes	36
CONCLUSÃO.....	44
REFERÊNCIAS.....	45

1 IA GENERATIVA: CRIATIVIDADE CODIFICADA

A Inteligência Artificial (IA) é uma disciplina científica focada na construção de sistemas capazes de realizar tarefas que tipicamente exigem inteligência humana. Essas tarefas abrangem um amplo espectro, incluindo aprender com a experiência, reconhecer padrões, resolver problemas, tomar decisões, compreender a linguagem e perceber o ambiente. A busca pela IA tem sido um objetivo de longa data, evoluindo através de vários paradigmas e abordagens ao longo de décadas.

Dentro do vasto cenário da IA, o Aprendizado de Máquina (Machine Learning — ML) emergiu como um paradigma poderoso que permitiu que os sistemas aprendessem diretamente dos dados sem serem explicitamente programados para cada cenário possível. Em vez de definir regras rígidas, os modelos de ML identificam padrões e relacionamentos dentro de grandes conjuntos de dados para fazer previsões ou decisões. Já o Aprendizado Profundo (Deep Learning — DL), um subcampo do ML, revolucionou ainda mais a área ao utilizar Redes Neurais Artificiais com múltiplas camadas — daí o termo “profundo”. Essas redes profundas são particularmente hábeis em aprender representações hierárquicas de dados, permitindo-lhes processar entradas complexas como imagens, áudio e texto com notável eficácia.

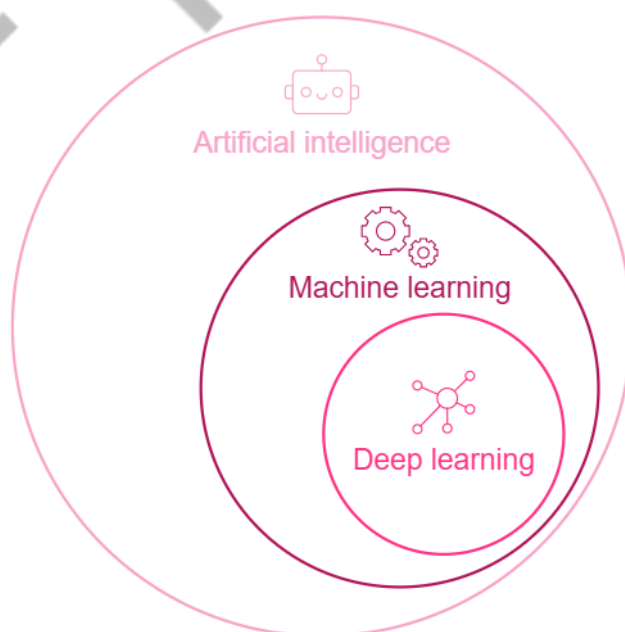


Figura 1 - Deep Learning é uma subárea de Machine Learning que, por sua vez, é uma subárea da Inteligência Artificial

Fonte: IA Responsável (2023), adaptado por FIAP (2026)

1.1 O que é um Modelo Generativo?

Historicamente, e ainda predominantemente em muitas aplicações, os modelos de IA e ML foram projetados para tarefas **discriminativas** (conforme mostra a figura “Acima é mostrado um modelo discriminativo. Abaixo, modelo generativo”). Modelos discriminativos são treinados para distinguir entre diferentes categorias ou para prever um valor específico com base nos dados de entrada. Sua principal função é mapear dados de entrada para um rótulo ou valor de saída, aprendendo os limites de decisão entre diferentes classes. Considere um filtro de spam: é um modelo discriminativo treinado para analisar o conteúdo de um e-mail e decidir se ele pertence à categoria “spam” ou “não spam”. Da mesma forma, um classificador de imagens aprende a analisar os pixels de uma imagem e atribuir-lhe um rótulo como “gato”, “cachorro” ou “pássaro”. Esses modelos são altamente eficazes em classificação, regressão e previsão, tarefas onde o objetivo é analisar dados existentes e tirar conclusões ou fazer previsões. Eles aprendem a probabilidade condicional de uma saída, dado uma entrada:

$$P(\text{Saída}|\text{Entrada})$$

A IA Generativa, no entanto, opera com um objetivo fundamentalmente diferente. Em vez de aprender a discriminar amostras de dados existentes em classes ou prever um resultado específico com base na entrada, os modelos generativos aprendem a criar novas instâncias de dados. Essas instâncias geradas são projetadas para se assemelhar aos dados nos quais o modelo foi treinado, capturando suas características subjacentes, padrões e estruturas. O objetivo não é classificar ou prever, mas sim **gerar** dados que poderiam plausivelmente ter se originado da mesma distribuição que os dados de treinamento. Esses modelos visam aprender a distribuição de probabilidade conjunta dos dados de entrada e saída $P(\text{Entradas}, \text{Saída})$, ou a probabilidade dos próprios dados $P(\text{Dados})$, o que lhes permite gerar novas entradas (e, potencialmente, saídas).

A essência da IA Generativa reside em sua capacidade de entender e replicar as propriedades intrínsecas de um conjunto de dados. Imagine treinar um modelo generativo em uma vasta coleção de fotografias de rostos. Um modelo discriminativo treinado nos mesmos dados poderia aprender a identificar se um rosto está sorrindo ou franzindo a testa, ou até mesmo reconhecer um indivíduo específico. Um modelo

generativo, no entanto, aprenderia as características típicas de rostos: o arranjo de olhos, nariz e boca, a textura da pele, variações na cor e estilo do cabelo, e poderia então sintetizar rostos inteiramente novos que não correspondem a nenhum indivíduo nos dados de treinamento, mas que parecem convincentemente reais.

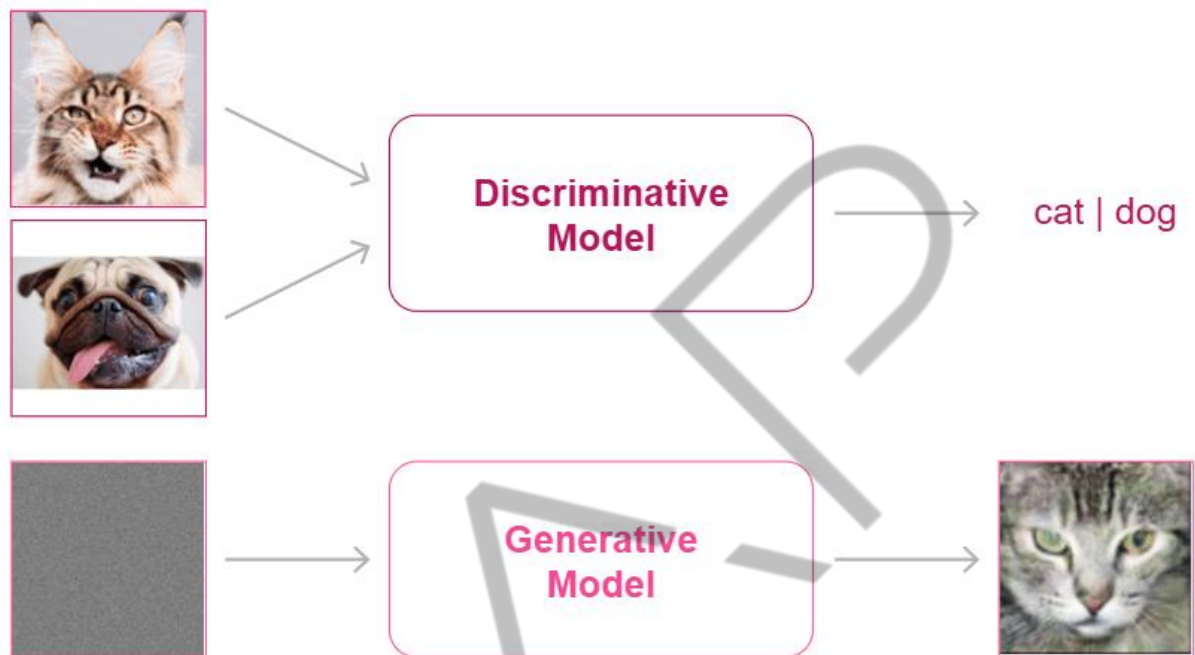


Figura 2 - Acima é mostrado um modelo discriminativo. Abaixo, modelo generativo
Fonte: ResearchGate (2020), adaptado por FIAP (2026)

1.2 Aplicações de Modelos Generativos

A capacidade de “geração” desses modelos é incrivelmente versátil e aplicável a uma infinidade de tipos de dados e domínios. A forma da saída gerada depende diretamente da natureza dos dados nos quais o modelo foi treinado e da arquitetura específica empregada.

Por exemplo, no domínio do texto, modelos generativos podem produzir passagens coerentes e contextualmente relevantes de linguagem humana. Treinados em enormes conjuntos de dados de livros, artigos, sites e conversas, esses modelos aprendem gramática, sintaxe, estilo, informações factuais e até nuances como tom e sentimento. Eles podem então ser solicitados a escrever redações, compor poesia, rascunhar e-mails, resumir documentos longos, traduzir texto entre idiomas e até mesmo gerar ficção científica. A capacidade de gerar texto com qualidade humana

tem profundas implicações para a criação de conteúdo, comunicação e processamento de informações.

No domínio visual, os modelos generativos têm alcançado resultados surpreendentes. Treinados em milhões, ou até mesmo bilhões, de imagens, eles podem criar imagens realistas de pessoas, paisagens, objetos e cenas abstratas que são indistinguíveis de fotografias reais para um observador casual. Além de criar imagens do zero, esses modelos também podem realizar tarefas complexas de manipulação de imagem. Isso inclui tarefas como: tradução de imagem para imagem (transformar uma foto em uma pintura em um estilo específico, ou mudar uma cena diurna para noturna), inpainting (preencher partes ausentes de uma imagem) e outpainting (estender uma imagem além de suas bordas originais). Modelos de text-to-image, que geram imagens com base em descrições em linguagem natural, abriram novas possibilidades para a expressão artística e a criação de conteúdo visual.

Áudio é outro domínio onde a IA Generativa está progredindo significativamente. Modelos podem ser treinados em bibliotecas de música, fala ou sons ambientais para gerar novos conteúdos de áudio. Isso inclui compor peças musicais originais em vários gêneros, sintetizar a fala humana realista em diferentes vozes e idiomas (text-to-speech) e criar efeitos sonoros. As aplicações variam desde a criação de música de fundo para vídeos e jogos até a geração de narrações e o desenvolvimento de assistentes virtuais personalizados.

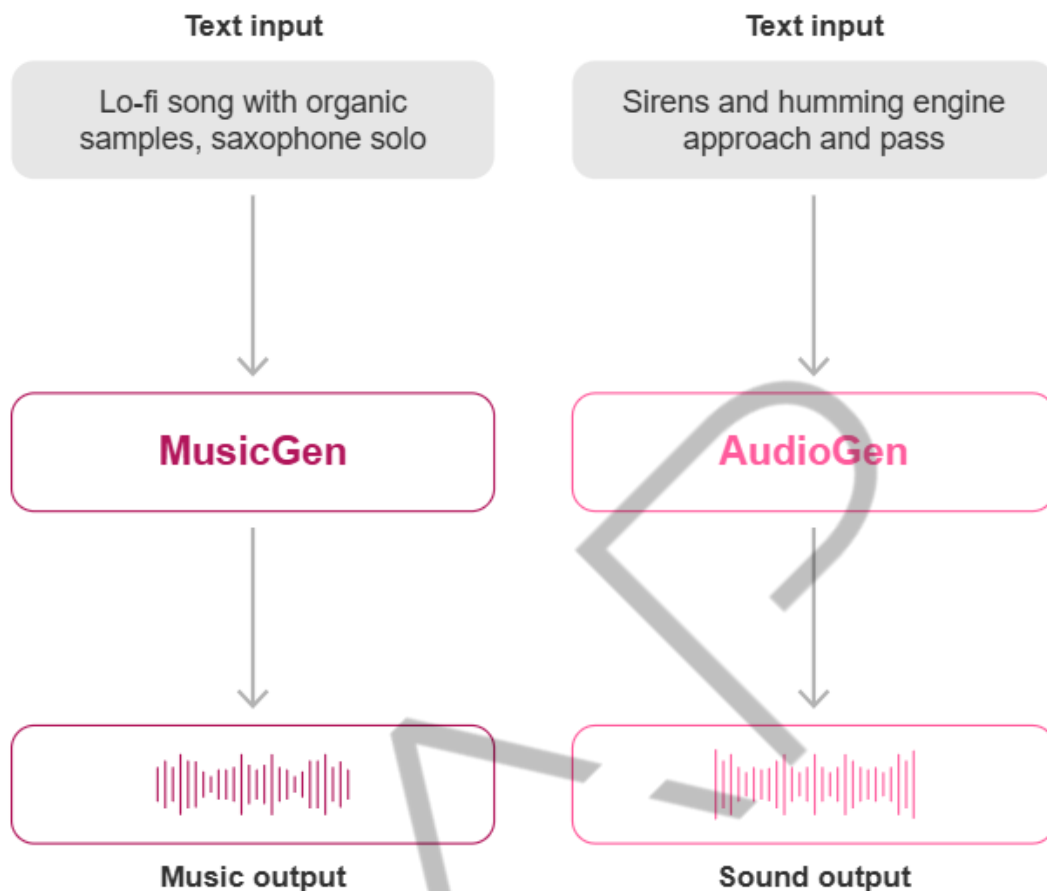


Figura 3 - Modelos generativos de geração de áudios
Fonte: Meta (2023), adaptado por FIAP (2026)

A geração de conteúdo de vídeo, embora computacionalmente mais intensiva, também está se tornando cada vez mais viável. Modelos generativos podem criar clipes de vídeo curtos, animar imagens estáticas ou até mesmo gerar sequências de vídeo com base em descrições de texto ou entradas de áudio. Essa tecnologia tem o potencial de revolucionar a produção cinematográfica, a animação e a criação de conteúdo visual dinâmico.

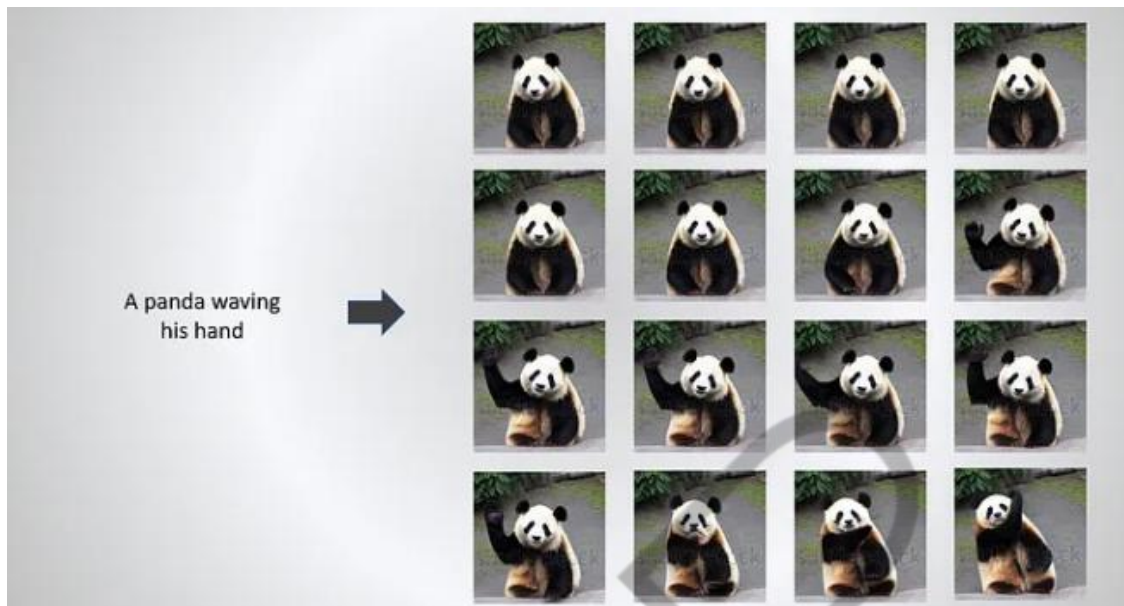


Figura 4 - Modelos generativos de geração de vídeo
Fonte: Medium – Introduction to VideoFusion (2023)

Além do conteúdo criativo, a IA Generativa também é uma poderosa ferramenta para gerar dados estruturados, incluindo código. Grandes Modelos de Linguagem (Large Language Models — LLMs), quando treinados em grandes repositórios de código-fonte, podem aprender padrões de programação, sintaxe e boas práticas. A partir disso, conseguem auxiliar os desenvolvedores sugerindo complementos de código, gerando trechos de código com base em descrições em linguagem natural da funcionalidade desejada ou até mesmo escrevendo funções ou classes inteiras. Essa capacidade tem o potencial de acelerar significativamente o desenvolvimento de software e já está sendo explorada em grandes centros tecnológicos, dando origem a um novo papel no desenvolvimento de sistemas chamado de Vibe Coding, que é o desenvolvimento colaborativo entre humano e a IA.

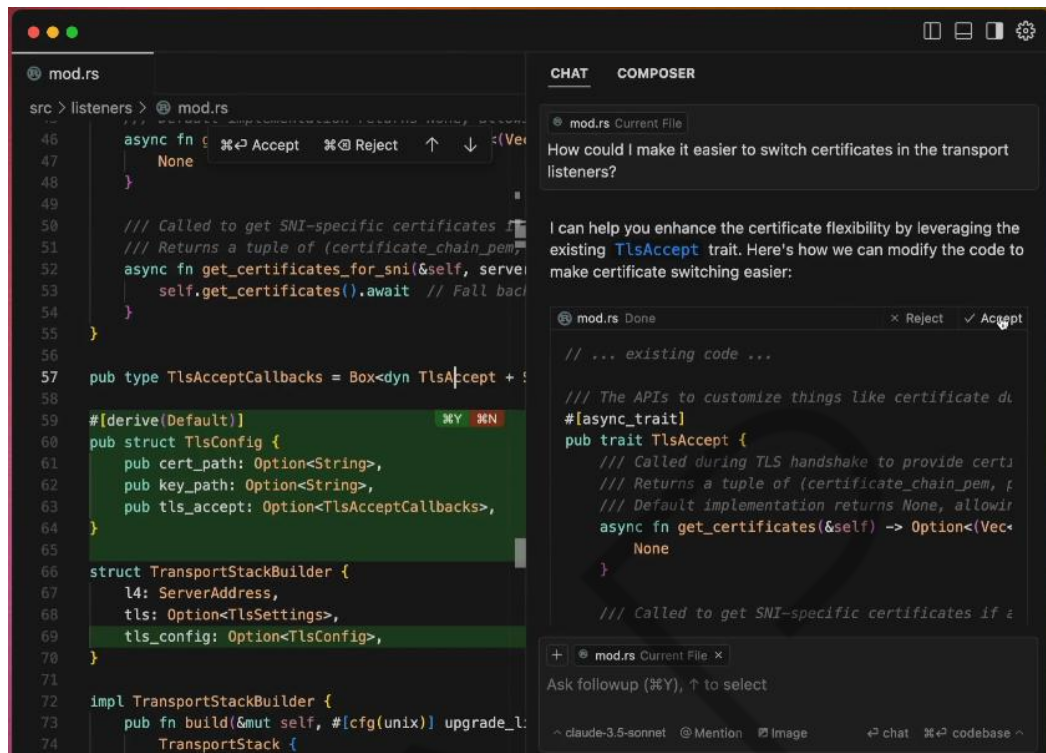


Figura 5 - Vibe Coding: humanos desenvolvendo códigos com auxílio de IA
Fonte: Gigazine (2025)

As aplicações se estendem ainda mais para domínios científicos e industriais. Na descoberta de medicamentos, modelos generativos podem propor novas estruturas moleculares com propriedades desejadas, potencialmente acelerando a identificação de novos compostos terapêuticos. Da mesma forma, na ciência de materiais, eles podem sugerir designs para novos materiais com características específicas. A IA Generativa também pode ser utilizada para aumento de dados, criando dados sintéticos de treinamento para outros modelos de IA, melhorando assim sua robustez e desempenho. Simulações realistas para treinar sistemas autônomos ou analisar fenômenos complexos também podem ser geradas.

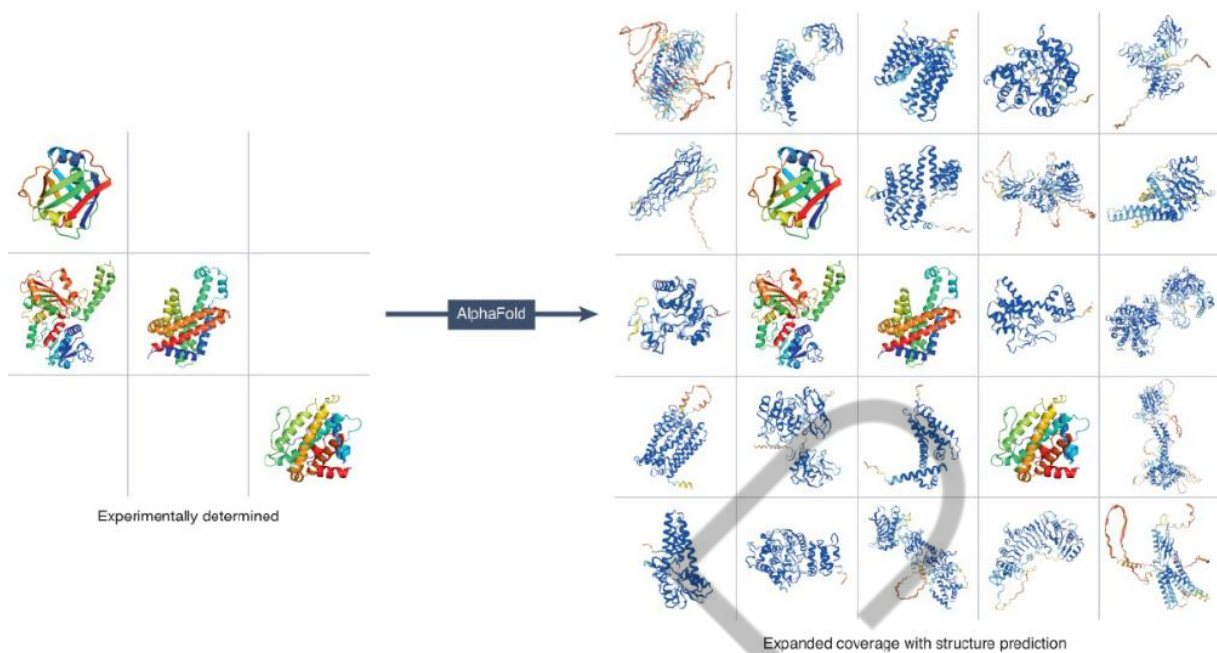


Figura 6 - Alphafold: Modelo generativo para gerar estruturas 3D de proteínas ainda desconhecidas
Fonte: Nature (2022)

1.3 História da IA Generativa

A capacidade de criar algo a partir de dados existentes tem sido um fascínio na Inteligência Artificial por décadas. Embora o termo “IA Generativa” tenha ganhado destaque recentemente, as ideias e técnicas subjacentes têm raízes profundas na história da Computação e da Inteligência Artificial. Esta seção aborda a evolução dos modelos generativos, desde os primeiros conceitos até as capacidades de ponta que vemos hoje.

Os primeiros esforços em IA que se assemelhavam à geração de conteúdo muitas vezes se baseavam em regras explícitas e estruturas predefinidas. A ideia era simular a criatividade ou a comunicação humana através de sistemas lógicos e procedimentos.

Um dos exemplos mais notáveis e precoces de geração baseada em regras — também conhecida como geração de conteúdo procedural inicial — é o programa ELIZA, criado por Joseph Weizenbaum em 1966. ELIZA simulava uma terapeuta rogeriana, respondendo a entradas do usuário com frases predefinidas e transformações simples baseadas em palavras-chave. Embora não “gerasse” conteúdo no sentido moderno de aprender distribuições de dados, ela criava a ilusão

de uma conversa gerada, manipulando e reestruturando a entrada do usuário de acordo com um conjunto de regras.

```
Welcome to

EEEEEE LL      IIII  ZZZZZZ  AAAAA
EE      LL      II    ZZ    AA  AA
EEEEEE LL      II    ZZ    AAAAAA
EE      LL      II    ZZ    AA  AA
EEEEEE LLLLLL  IIII  ZZZZZZ  AA  AA

Eliza is a mock Rogerian psychotherapist.
The original program was described by Joseph Weizenbaum in 1966.
This implementation by Norbert Landsteiner 2005.

ELIZA: Is something troubling you ?
YOU:   Men are all alike.
ELIZA: What is the connection, do you suppose ?
YOU:   They're always bugging us about something or other.
ELIZA: Can you think of a specific example ?
YOU:   Well, my boyfriend made me come here.
ELIZA: Is it important to you that your boyfriend made you come here ?
YOU:   He says I'm depressed much of the time.
ELIZA: I am sorry to hear that you are depressed.
YOU:   It's true. I am unhappy.
ELIZA: Can you explain what made you unhappy ?
YOU:
```

Figura 7 - ELIZA

Fonte: Medium – Engenharia de Prompt – História dos modelos de linguagem – Parte 3 (2024)

Geradores de conteúdo procedural em jogos de computador e outras aplicações também representam uma forma inicial de geração. Eles usavam algoritmos e regras para criar automaticamente terrenos, níveis, texturas ou outros elementos de jogo, em vez de serem criados manualmente. Isso permitia mundos de jogo maiores e mais variados, embora a complexidade e a novidade fossem limitadas pelas regras programadas.

Uma abordagem mais fundamentada em dados surgiu com os modelos estatísticos, quando se destacaram as Cadeias de Markov. Uma Cadeia de Markov é um modelo matemático que prevê a probabilidade do próximo estado (como a próxima palavra em uma frase) com base apenas no estado atual (a palavra anterior ou um pequeno histórico de palavras). Modelos de linguagem estatísticos, como modelos N-grama (que consideram sequências de N palavras), foram amplamente utilizados no Processamento de Linguagem Natural (PLN) no final do século XX para tarefas como reconhecimento de fala e, em menor grau, para geração de texto simples.

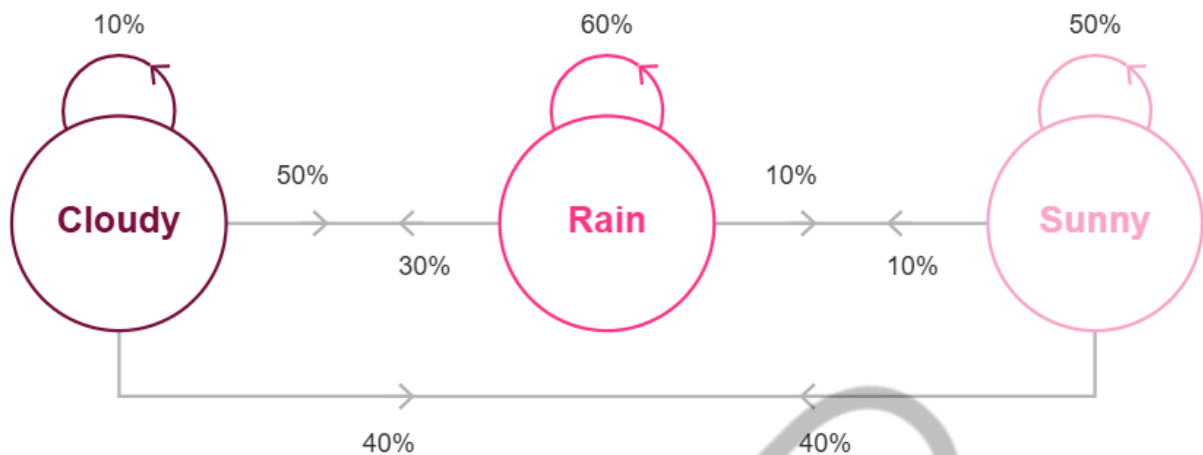


Figura 8 - Cadeias de Markov

Fonte: Medium – Act Like Markov (2021), adaptado por FIAP (2026)

Esses modelos podiam gerar sequências de texto que tinham alguma coerência local, mas careciam de compreensão contextual de longo alcance ou de uma estrutura global que considerasse toda a janela de contexto das conversas. A geração era essencialmente uma amostragem probabilística baseada em frequências observadas nos dados de treinamento.

A virada do século XXI e, em particular, a década de 2010, testemunhou uma revolução na IA impulsionada pelo Aprendizado Profundo. A capacidade das Redes Neurais Profundas de aprender representações complexas e hierárquicas diretamente dos dados abriu novas e poderosas possibilidades para a geração de dados.

O aumento do poder computacional — especialmente com o advento das General Purpose Graphic Processing Units GP (GPUs) — e a disponibilidade de grandes conjuntos de dados tornaram possível treinar Redes Neurais com muitas camadas. Arquiteturas como Redes Neurais Recorrentes (RNNs), particularmente as suas variações como Long Short-Term Memory (LSTM), mostraram-se eficazes na modelagem de dados sequenciais como texto e áudio, permitindo a geração de sequências mais longas e coerentes do que os modelos estatísticos anteriores, baseados em Cadeias de Markov.

IA Generativa: Criatividade Codificada

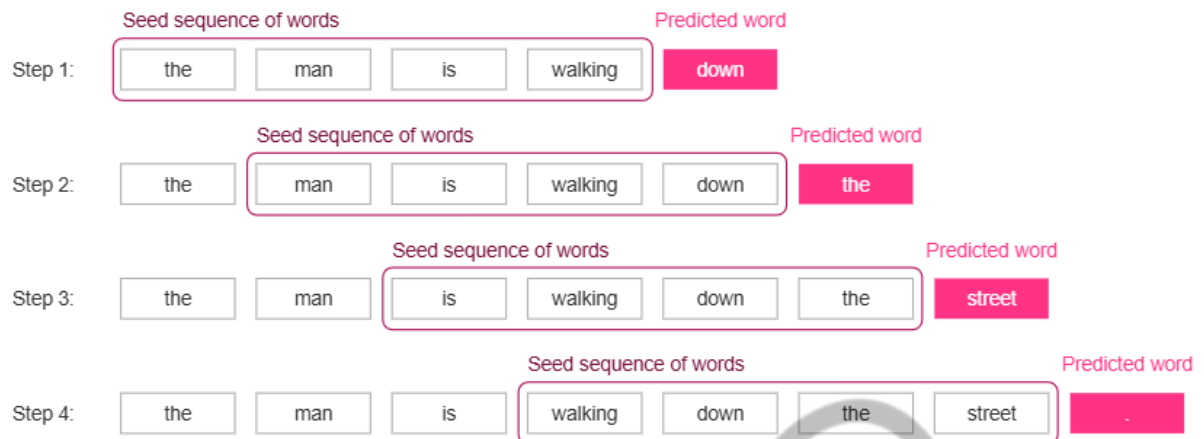


Figura 9 - Long Short-Term Memory Networks predizendo a próxima palavra com base nas palavras anteriores

Fonte: Medium – Text generation using LSTM (2021), adaptado por FIAP (2026)

Os Autoencoders são Redes Neurais treinadas para copiar sua entrada para sua saída. Eles consistem em um codificador que comprime a entrada em uma representação de menor dimensão (o “espaço latente”) e um decodificador que reconstrói a entrada a partir dessa representação latente. Embora originalmente utilizados para redução de dimensionalidade e denoising, eles formaram a base para modelos generativos.

Os Variational Autoencoders (VAEs), introduzidos por Diederik Kingma e Max Welling em 2013, adicionaram uma camada probabilística a essa estrutura. Em vez do codificador gerar um único ponto no espaço latente, ele gera parâmetros (média e variância) de uma distribuição (geralmente Gaussiana) no espaço latente. A geração de novos dados em um VAE envolve a amostragem de pontos desse espaço latente aprendido e a passagem desses pontos pelo decodificador. Os VAEs foram um dos primeiros modelos de Aprendizado Profundo capazes de gerar dados novos e variados, embora as amostras geradas pudessem ser um pouco borradas em comparação com métodos posteriores.

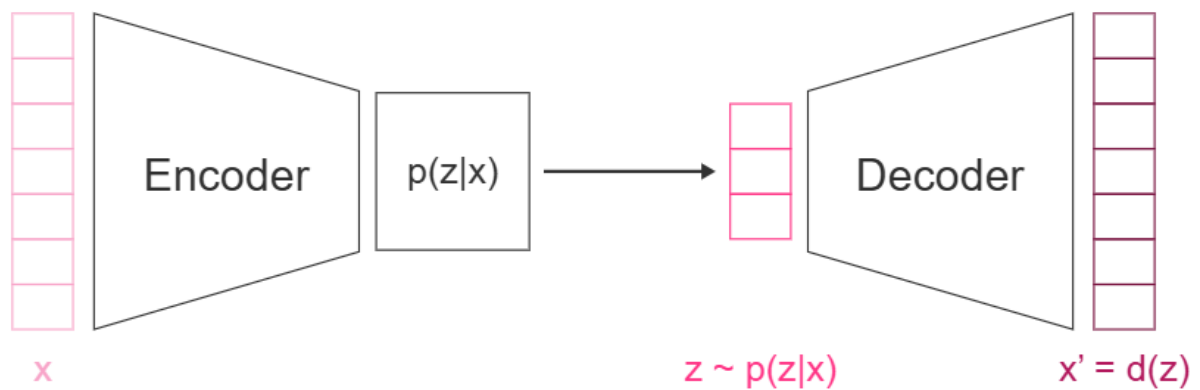


Figura 10 - Variational Autoencoders

Fonte: Introduction to Variational Autoencoders and their applications (2022), adaptado por FIAP (2026)

Introduzidas por Ian Goodfellow e colegas em 2014, as Redes Adversárias Generativas (GANs) representaram uma ideia revolucionária na geração de dados. Uma GAN consiste em duas Redes Neurais competindo entre si: um Gerador e um Discriminador. O Gerador tenta criar dados falsos que se pareçam com os dados reais, enquanto o Discriminador tenta distinguir entre dados reais e dados falsos criados pelo Gerador. As duas redes são treinadas simultaneamente em um jogo de soma zero. O Gerador melhora na criação de dados realistas para enganar o Discriminador, e o Discriminador melhora ao detectar dados falsos. Esse processo adversarial leva o Gerador a produzir dados cada vez mais convincentes.

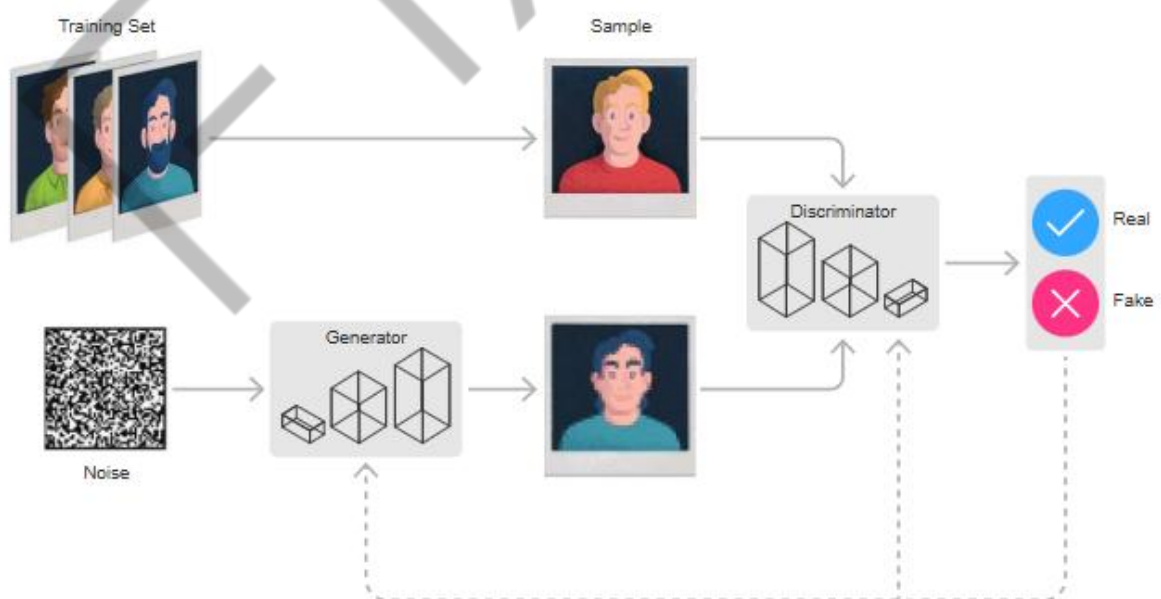


Figura 11 - GANs

Fonte: Medium – GANs: Simply Explained! (2023), adaptado por FIAP (2025)

As GANs foram particularmente bem-sucedidas na geração de imagens de alta qualidade e realistas, resultando em avanços notáveis em tarefas como síntese de imagem, transferência de estilo e super-resolução. No entanto, o treinamento de GANs pode ser instável (o chamado “modo colapso”, onde o gerador produz pouca variedade) e difícil de controlar.

A introdução da arquitetura Transformer em 2017 por Vaswani *et al.* marcou um ponto de virada, especialmente no PLN. A chave para os Transformers é o mecanismo de “atenção”, que permite que o modelo pondere a importância de diferentes partes da entrada ao processar uma sequência, superando as limitações de memória de longo prazo das LSTMs.

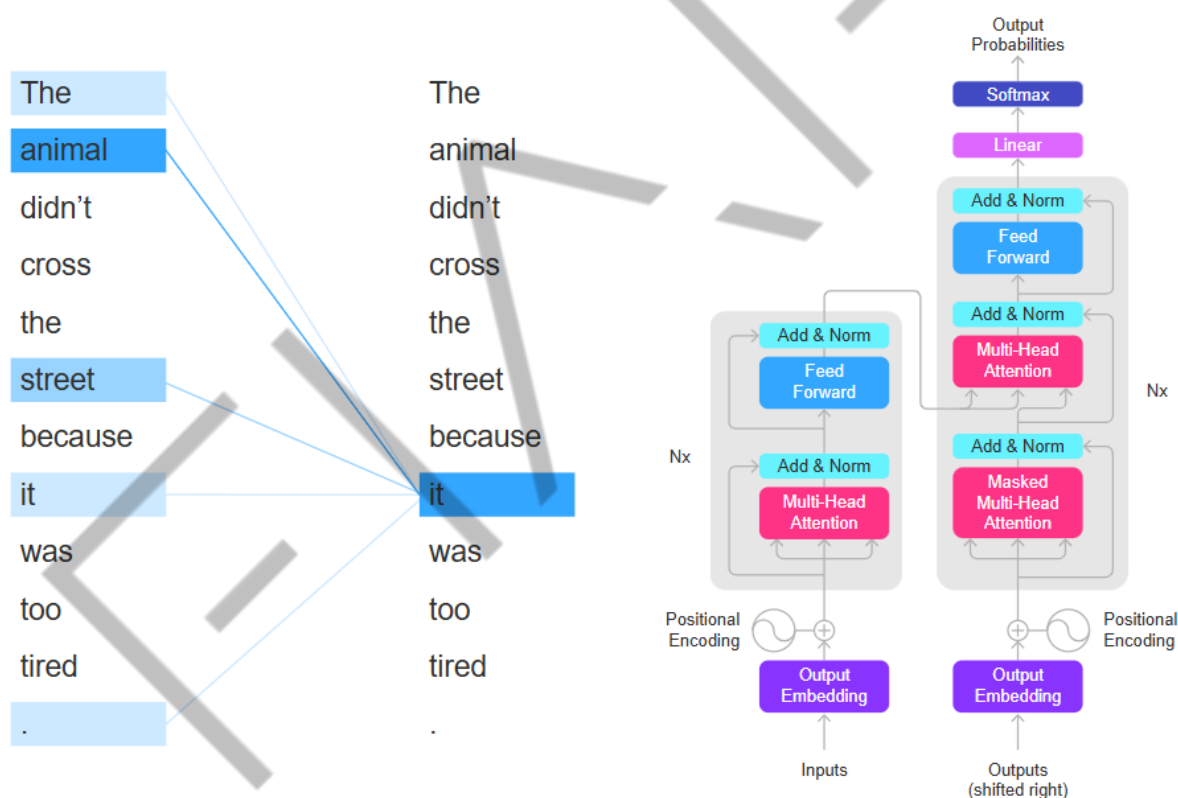


Figura 12 - Arquitetura Transformer e Mecanismo de Atenção

Fonte: Dio – Além das Sequências Curtas: Como a IA está Aprendendo a Lidar com Contextos Extensos (2024), adaptado por FIAP (2025)

Modelos baseados em Transformer, como o Generative Pre-trained Transformer (GPT) do OpenAI, demonstraram capacidades sem precedentes na geração de texto coerente e contextualmente relevante em larga escala. O conceito de pré-treinar um grande modelo em um vasto corpus de texto e depois ajustá-lo para tarefas específicas (ou usá-lo diretamente para geração zero-shot/few-shot) levou à ideia de “Foundation Models” — modelos grandes e pré-treinados que podem ser

IA Generativa: Criatividade Codificada

adaptados para uma ampla gama de tarefas a jusante. Essa arquitetura e abordagem de treinamento se tornaram essenciais para a era moderna da IA Generativa, estendendo-se rapidamente para outros domínios além do texto.

Nos últimos anos, observou-se uma explosão de pesquisa e desenvolvimento em IA Generativa, impulsionada por modelos baseados em Transformers e em novas arquiteturas.

Modelos como as várias iterações do GPT (GPT-2, GPT-3, GPT-4) continuaram a empurrar os limites da geração de texto, exibindo capacidades notáveis em compreensão e geração de linguagem natural.

Paralelamente, modelos de texto para imagem como DALL·E (OpenAI), Midjourney e Stable Diffusion (Stability AI) revolucionaram a criação da arte digital e imagens, permitindo que os usuários gerassem imagens complexas e criativas a partir de descrições textuais simples. Esses modelos frequentemente se baseiam em arquiteturas Transformer (ou variantes) e, no caso de Stable Diffusion e outros, em modelos de difusão latente.

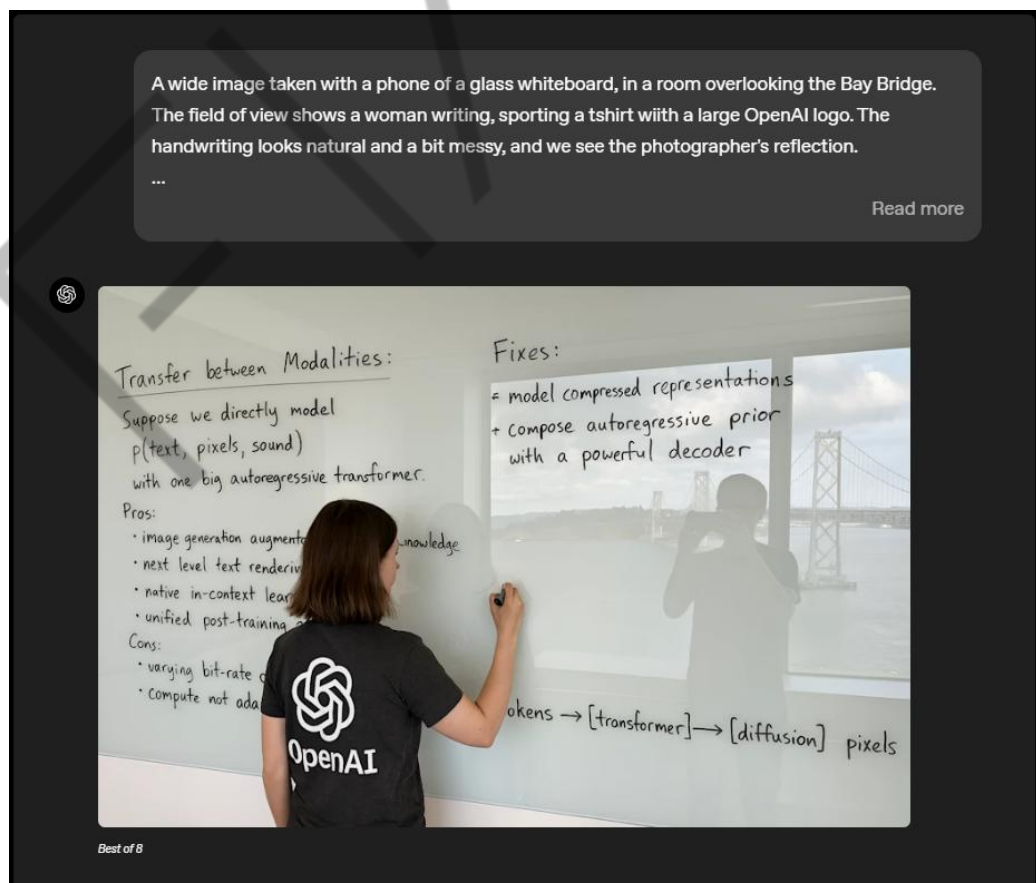


Figura 13 - Modelo de geração de imagens da OpenAI
Fonte: TecMundo (2025)

Geração multimodal e entre domínios: um desenvolvimento chave é a capacidade crescente dos modelos de trabalhar com e gerar conteúdo em múltiplas modalidades (texto, imagem, áudio, vídeo) ao mesmo tempo. Modelos como o DALL·E original e o GPT-4 com suas capacidades multimodais são exemplos dessa tendência. Isso permite tarefas como gerar legendas para imagens, responder a perguntas sobre o conteúdo de imagens, ou gerar vídeos a partir de descrições de texto e áudio. A pesquisa está se movendo rapidamente em direção a modelos unificados que podem entender e gerar qualquer combinação de modalidades.



Figura 14 - Modelo Multimodais
Fonte: Reddit (2025)

A geração entre domínios — como a geração de código a partir de texto, exemplificada por ferramentas como o AlphaCode, do DeepMind, e o GitHub Copilot, que utiliza modelos da OpenAI —, ou mesmo a geração de estruturas moleculares, também se tornou uma área ativa, mostrando que os princípios da IA Generativa são aplicáveis muito além das mídias tradicionais.

A história da IA Generativa é uma narrativa de evolução contínua, passando de sistemas baseados em regras para modelos estatísticos, e finalmente para as

poderosas e versáteis arquiteturas de Aprendizado Profundo que definem o campo hoje. Cada era foi construída sobre a anterior, superando limitações e desbloqueando novas capacidades, levando-nos ao ponto atual onde a IA pode não apenas analisar o mundo, mas também criar novas facetas dele.

1.4 Foundation Models e Ecossistema

A era atual da IA Generativa é amplamente definida pelo surgimento e domínio dos chamados “Foundation Models”. Esses modelos, tipicamente de grande escala e treinados em grandes e diversos conjuntos de dados, servem como base para uma ampla gama de tarefas generativas e discriminativas. No centro de muitos desses modelos está uma arquitetura revolucionária: o Transformer.

1.4.1 O advento dos Transformers

Introduzida em 2017 no artigo “Attention Is All You Need”, a arquitetura Transformer transformou o Processamento de Linguagem Natural e rapidamente se tornou a base para muitos modelos de ponta em outros domínios, como visão computacional. Diferentemente das arquiteturas recorrentes (como RNNs e LSTMs) que processam dados sequencialmente, o Transformer processa sequências em paralelo, tornando o treinamento em grandes conjuntos de dados muito mais eficiente e permitindo que o modelo capture dependências de longo alcance de forma mais eficaz.

O componente mais crucial do Transformer é o mecanismo de **atenção** (ou “self-attention”). A atenção permite que o modelo pondere a importância de diferentes partes da sequência de entrada ao processar um elemento específico. Por exemplo, ao processar a palavra “banco” em uma frase, o mecanismo de atenção pode dar mais peso a palavras como “rio” ou “dinheiro” para entender o sentido correto da palavra no contexto. Isso contrasta com modelos anteriores que tinham uma janela de contexto limitada ou lutavam para manter informações relevantes ao decorrer de longas sequências. A atenção permite que o modelo crie conexões diretas entre quaisquer duas posições na sequência de entrada, independentemente de sua distância.

A arquitetura Transformer consiste tipicamente em um codificador e um decodificador (embora modelos generativos como o GPT utilizem apenas a parte do decodificador). O codificador processa a sequência de entrada, e o decodificador usa a saída do codificador (e as saídas geradas anteriormente) para produzir a sequência de saída. Ambos os componentes são construídos a partir de múltiplas camadas idênticas, cada uma contendo subcamadas de atenção multi-head e Redes Neurais feed-forward.

O processo de treinamento de modelos baseados em Transformer geralmente envolve duas fases principais: pré-treinamento e ajuste fino (fine-tuning). No pré-treinamento, o modelo é treinado em uma tarefa de larga escala e auto-supervisionada em um vasto e diverso conjunto de dados. Para modelos de linguagem, a tarefa comum é a modelagem de linguagem, onde o modelo aprende a prever a próxima palavra em uma sequência (como em modelos GPT) ou a preencher palavras mascaradas (como em modelos BERT). Este pré-treinamento permite que o modelo aprenda representações ricas e gerais da linguagem (ou de outros tipos de dados).

Após o pré-treinamento, o modelo pode sofrer um ajuste fino com conjuntos de dados menores e rotulados para realizar tarefas específicas. Por exemplo, um modelo de linguagem pré-treinado pode ser ajustado para classificação de sentimento, tradução automática ou resposta a perguntas. O ajuste fino adapta o conhecimento geral adquirido durante o pré-treinamento para o domínio ou tarefa específica.

1.4.2 Foundation Models

O termo “Foundation Models”, popularizado por pesquisadores da Universidade de Stanford, refere-se a modelos de grande escala, pré-treinados em dados amplos e diversos, que podem ser adaptados (por meio de ajuste fino, prompt engineering ou outras técnicas) para uma ampla variedade de tarefas a jusante, sem a necessidade de treinar um novo modelo do zero para cada tarefa.

O que torna um modelo “fundacional”?

- **Escala:** eles são tipicamente muito grandes em termos de número de parâmetros (bilhões ou trilhões).

- **Generalidade:** são treinados em conjuntos de dados extremamente amplos e diversos, permitindo-lhes aprender uma vasta gama de conceitos, padrões e habilidades.
- **Capacidade de adaptação:** podem ser adaptados para muitas tarefas diferentes, tanto generativas quanto discriminativas, com esforço relativamente menor em comparação com o treinamento de modelos específicos para cada tarefa.

Existem distinções importantes entre diferentes formas de usar ou adaptar modelos fundacionais:

- **Modelos pré-treinados:** o modelo na sua forma original após a fase de pré-treinamento em larga escala. Ele possui conhecimento geral, mas pode não ser otimizado para uma tarefa específica.
- **Modelos ajustados (fine-tuned):** o modelo pré-treinado que passou por uma fase adicional de treinamento em um conjunto de dados menor e específico para uma tarefa. Isso melhora seu desempenho nessa tarefa particular.
- **Modelos ajustados para instrução (instruction-tuned):** uma forma especializada de ajuste fino onde o modelo é treinado para seguir instruções dadas em linguagem natural. Isso os torna mais úteis para prompts abertos e para realizar uma variedade de tarefas sem a necessidade de ajuste fino adicional para cada uma. Muitos dos modelos de chat populares são modelos ajustados por instrução.

“Foundation Models” populares incluem:

- **Série GPT (OpenAI):** modelos autorregressivos baseados em Transformer, amplamente conhecidos por suas capacidades de geração de texto.
- **LLaMA (Meta AI):** uma família de Modelos de Linguagem Grandes de código aberto que se tornou popular na comunidade de pesquisa.
- **Claude (Anthropic):** modelos de linguagem desenvolvidos com foco em serem úteis, honestos e inofensivos.

- **Gemini (Google DeepMind):** uma família de modelos multimodais projetados para serem altamente capazes em diversos domínios.
- **Outros modelos:** existem muitos outros modelos fundacionais desenvolvidos por diversas organizações e pela comunidade de código aberto.

Visão geral do ecossistema:

O desenvolvimento e a aplicação de modelos fundacionais impulsionaram um ecossistema vibrante de pesquisa, desenvolvimento e implantação. Várias organizações estão na vanguarda deste campo:

- **Hugging Face:** tornou-se um hub central para a comunidade de IA, fornecendo um vasto repositório de modelos pré-treinados (incluindo muitos modelos fundacionais), conjuntos de dados e ferramentas (a biblioteca Transformers) que facilitam o acesso e a utilização desses modelos. Eles promovem ativamente a ciência aberta em IA.
- **OpenAI:** uma organização de pesquisa e implantação de IA responsável pelo GPT, DALL·E e outros modelos proeminentes. Têm sido pioneira no desenvolvimento de modelos de larga escala e suas aplicações.
- **Anthropic:** uma empresa de segurança de IA que desenvolve modelos de linguagem, como a família Claude, com um forte foco em alinhamento e segurança.
- **Meta AI (anteriormente Facebook AI):** a equipe de pesquisa de IA da Meta, responsável pelo desenvolvimento de modelos como LLaMA, tem contribuído significativamente para a pesquisa em diversas áreas da IA incluindo modelos generativos.
- **Google DeepMind:** a equipe de pesquisa de IA do Google, conhecida por seus avanços em aprendizado por reforço e pelo desenvolvimento de modelos como AlphaFold e a família Gemini, tem como foco capacidades multimodais e raciocínio avançado.

Este ecossistema em rápida evolução, impulsionado por avanços em arquiteturas como o Transformer e o desenvolvimento contínuo de modelos fundacionais maiores e mais capazes, está no centro da revolução da IA Generativa.

A colaboração — e competição — entre essas entidades e a comunidade de código aberto continua a moldar o futuro da área.

1.5 Implicações Éticas, Legais e Sociais

O rápido avanço e a crescente adoção da IA Generativa trazem consigo uma série complexa de questões éticas, legais e sociais. A capacidade de gerar conteúdo em larga escala e com alta fidelidade levanta preocupações importantes sobre justiça, verdade, propriedade e o impacto na sociedade e no mercado de trabalho. Enfrentar esses desafios é fundamental para assegurar que a IA Generativa seja desenvolvida e utilizada de forma responsável e benéfica.

Um dos obstáculos mais significativos que enfrentamos com a IA Generativa é a questão do viés e da justiça. Modelos generativos aprendem intrinsecamente com os dados nos quais são treinados. Se esses dados contêm vieses — refletindo desigualdades sociais existentes, estereótipos ou representações desproporcionais de certos grupos na sociedade —, o modelo generativo não apenas aprenderá esses vieses, mas também poderá, inadvertidamente, amplificá-los no conteúdo que gera. O viés nos dados de treinamento e sua consequente amplificação podem se manifestar de várias formas preocupantes. Um modelo de texto treinado com dados da internet, por exemplo, pode aprender e reproduzir estereótipos prejudiciais relacionados a gênero, raça, origem étnica ou outras características demográficas. Da mesma forma, um modelo de geração de imagens pode gerar imagens que sub-representam certos grupos demográficos ou retratá-los de maneira estereotipada e preconceituosa. Considere um cenário onde um modelo de texto-para-imagem é treinado predominantemente com imagens de médicos que são homens brancos; esse modelo pode subsequentemente ter dificuldade em gerar imagens de médicas mulheres ou médicos de outras etnias quando solicitado, ou pode associar certas profissões a um gênero ou raça específica de forma injusta.

A amplificação do viés ocorre porque o modelo não apenas reflete as correlações estatísticas encontradas nos dados, mas as utiliza como base para criar novas amostras, potencialmente solidificando e até agravando desigualdades e injustiças sociais através do conteúdo gerado e disseminado. A comunidade de pesquisa e desenvolvimento está trabalhando ativamente em métodos para detectar

e mitigar o viés em modelos generativos. Isso inclui a curadoria cuidadosa de dados, selecionando e equilibrando os conjuntos de treinamento para garantir representação justa e reduzir a presença de vieses prejudiciais. Também envolve o desenvolvimento de técnicas de treinamento que penalizem a geração de conteúdo enviesado e a criação de métricas e benchmarks robustos para avaliar a presença de viés nas saídas do modelo. Além disso, explorar controles que permitam aos usuários especificar atributos desejados na geração pode promover a diversidade (embora seja reconhecido que isso também pode ser usado de forma maliciosa), e técnicas de pós-processamento podem ser aplicadas para ajustar ou filtrar a saída gerada e reduzir o viés. A mitigação de viés é um desafio contínuo que requer atenção constante em todo o ciclo de vida do desenvolvimento e implantação do modelo.

Com as capacidades de geração de conteúdo realista, surge a preocupação com deepfakes e desinformação. A capacidade da IA Generativa de criar conteúdo sintético altamente realista, como imagens, áudios e vídeos que parecem autênticos, levanta sérias preocupações sobre a disseminação intencional ou não intencional de deepfakes e informações falsas. Deepfakes são mídias sintéticas nas quais a semelhança de uma pessoa em uma imagem ou vídeo existente é substituída pela de outra pessoa. Embora possam ter usos legítimos em entretenimento ou criação artística, deepfakes maliciosos representam uma ameaça séria. Eles podem ser usados para criar vídeos falsos e convincentes de figuras públicas dizendo ou fazendo coisas que nunca fizeram, com o objetivo de manipular a opinião pública, difamar indivíduos ou espalhar desinformação em larga escala. A IA Generativa também pode ser empregada para gerar artigos de notícias falsas, postagens em mídias sociais ou outros conteúdos de texto que pareçam autênticos e confiáveis, dificultando a distinção entre o real e o sintético.

A crescente facilidade na criação de deepfakes de alta qualidade representa uma ameaça significativa à confiança na mídia digital e pode ter sérias consequências sociais e políticas, minando a base da informação compartilhada. A luta contra deepfakes e desinformação gerada por IA envolve uma combinação de abordagens tecnológicas e sociais. Isso inclui o desenvolvimento de modelos de IA discriminativos especificamente treinados para identificar artefatos ou inconsistências sutis em mídias sintéticas que não estão presentes em mídias reais. Explorar métodos para “marcar” o conteúdo gerado por IA, seja de forma invisível (marca d'água digital) ou visível, ou

criar sistemas para rastrear a origem e as modificações de mídias digitais (proveniência) são estratégias importantes. A educação midiática, informando o público sobre a existência e os perigos de deepfakes e desinformação gerada por IA, e promovendo o pensamento crítico e a verificação de fatos, é essencial. Além disso, plataformas de mídias sociais e outras plataformas online precisam implementar e fazer cumprir políticas claras para rotular, sinalizar ou remover conteúdos sintéticos enganosos. Esta é uma corrida armamentista tecnológica contínua, pois as técnicas para criar deepfakes e desinformação continuam a evoluir, exigindo inovação constante nas estratégias de detecção e mitigação.

A criação de conteúdo por máquinas também levanta questões complexas e ainda em evolução sobre propriedade e direitos autorais, enquadrando-se no domínio da propriedade intelectual. Se um modelo de IA Generativa cria uma imagem, uma peça musical, um texto literário ou até mesmo código de software, quem detém os direitos autorais sobre essa obra? É o desenvolvedor do modelo que criou a ferramenta? É o usuário que forneceu o prompt ou as instruções para a geração? É o proprietário dos vastos conjuntos de dados de treinamento que alimentaram o modelo? Ou, hipoteticamente, a própria IA, embora a IA não seja atualmente reconhecida como uma entidade legal capaz de possuir direitos autorais? As leis de propriedade intelectual existentes foram formuladas em um tempo onde a criação era exclusivamente uma atividade humana e, portanto, não se encaixam perfeitamente na realidade da criação por máquina. Diferentes jurisdições em todo o mundo estão começando a abordar essa questão através de decisões judiciais e propostas legislativas, mas ainda há muita incerteza legal e falta de consenso global.

Além disso, surge a questão de saber se o conteúdo gerado por IA pode, por si só, infringir os direitos autorais dos dados de treinamento. Se um modelo aprender de forma excessiva e replicar o estilo ou elementos específicos de obras protegidas por direitos autorais dentro de seu conjunto de treinamento (como, por exemplo, aprendendo excessivamente de um artista visual específico), a saída gerada poderia ser considerada uma obra derivada que infringe os direitos do artista original. A distinção entre “aprendizagem” e “cópia”, no contexto de modelos de larga escala treinados em vastos conjuntos de dados, é um desafio legal e técnico significativo.

Em resposta a esses e outros desafios, governos e órgãos reguladores em todo o mundo estão começando a considerar ativamente como regular a IA Generativa

para mitigar riscos e garantir o uso responsável e benéfico. A União Europeia, por exemplo, está na vanguarda com sua proposta de Lei de IA, que adota uma abordagem baseada em risco, classificando os sistemas de IA com base em seu potencial de causar dano e impondo obrigações correspondentes. Modelos generativos de uso geral, como grandes modelos de linguagem, estão sendo considerados sob esta estrutura, com requisitos potenciais relacionados à transparência (informar quando o conteúdo é gerado por IA), gerenciamento de riscos (avaliar e mitigar riscos de viés, desinformação etc.) e governança de dados. Nos Estados Unidos, a abordagem tem sido mais focada em diretrizes, estruturas voluntárias e ações executivas, embora exista discussões contínuas sobre a necessidade e a forma de regulamentações mais formais. Globalmente, há um esforço crescente para desenvolver normas e padrões internacionais para a IA, reconhecendo sua natureza transnacional.

A regulamentação da IA Generativa é um desafio complexo, pois é preciso encontrar um equilíbrio delicado entre a mitigação de riscos sérios e a promoção da inovação e dos benefícios potenciais da tecnologia. As políticas estão em constante evolução à medida que a tecnologia avança e seus impactos sociais e econômicos se tornam mais claros.

Finalmente, a proliferação da IA Generativa levanta questões profundas sobre o futuro do trabalho e a relação em constante mudança entre humanos e máquinas criativas. Uma das perspectivas enxerga a IA Generativa como uma ferramenta poderosa para **aumentar as capacidades humanas**. Nesta narrativa de aumento, a IA Generativa atua como um copiloto ou assistente que capacita os humanos a serem mais produtivos, criativos e eficientes. Artistas podem utilizar modelos de texto-para-imagem como co-criadores para explorar novas ideias visuais, escritores podem usar modelos de linguagem para superar o bloqueio criativo, gerar rascunhos ou refinar seu texto, e programadores podem utilizar as ferramentas de geração de código para acelerar o desenvolvimento e focar em problemas de design de alto nível. Nesta visão, a IA Generativa libera os humanos para se concentrarem em tarefas de nível superior que exigem criatividade estratégica, julgamento crítico, empatia e habilidades interpessoais — áreas onde os humanos ainda possuem uma vantagem distintiva.

A perspectiva oposta é a da **substituição**, onde a IA Generativa poderia automatizar tarefas que antes eram realizadas exclusivamente por humanos, levando

a um potencial perda de empregos em indústrias criativas, jornalismo, programação, design gráfico e outras áreas onde a geração de conteúdo é uma parte central do trabalho. A realidade, como frequentemente acontece com automação e avanços tecnológicos, provavelmente será uma combinação de ambos os cenários, com algumas tarefas sendo automatizadas, outras sendo transformadas e novas funções e indústrias emergindo. Abordar o impacto social da IA Generativa no mercado de trabalho exigirá investimentos significativos em requalificação e educação para ajudar os trabalhadores a se adaptarem a um cenário de emprego em mudança e adquirir as habilidades necessárias para colaborar efetivamente com sistemas de IA.

As considerações éticas, legais e sociais da IA Generativa não devem ser tratadas como desafios secundários ou detalhes de menor importância; elas são fundamentais para o desenvolvimento responsável, a implantação segura e o impacto positivo dessa tecnologia poderosa na sociedade. À medida que a IA Generativa continua a evoluir em capacidade e ubiquidade, o diálogo contínuo, a pesquisa dedicada e a colaboração entre pesquisadores, desenvolvedores, formuladores de políticas, usuários e a sociedade em geral serão absolutamente essenciais para navegar nesses desafios complexos e moldar um futuro onde a IA Generativa contribua para o bem da humanidade, mitigando seus riscos potenciais.

1.6 Desafios e Limitações

Apesar dos avanços notáveis e do potencial transformador da IA Generativa, é fundamental reconhecer que essa tecnologia ainda enfrenta desafios significativos e possui limitações inerentes. Compreender essas dificuldades é essencial para pesquisadores, desenvolvedores, usuários e formuladores de políticas, pois permite alinhar as expectativas e trabalhar em direção a sistemas mais robustos, confiáveis e seguros.

Um dos obstáculos mais proeminentes é o custo computacional associado ao treinamento e à execução de modelos generativos de larga escala. O pré-treinamento de modelos fundacionais, como os grandes modelos de linguagem ou modelos de difusão de imagem, requer uma enorme quantidade de poder de processamento, geralmente utilizando milhares de GPUs ou TPUs por semanas ou meses. Isso se traduz em um consumo de energia substancial e custos financeiros elevados,

tornando o desenvolvimento de modelos de ponta acessível apenas para grandes organizações com recursos significativos. A escalabilidade também é um desafio; à medida que os modelos se tornam maiores para alcançar melhor desempenho, os requisitos computacionais aumentam exponencialmente. A inferência, ou seja, a execução do modelo para gerar novas amostras após o treinamento, também pode ser computacionalmente intensiva, afetando a velocidade e o custo de implantação em larga escala. A busca por arquiteturas e algoritmos mais eficientes em termos computacionais e energéticos é uma área ativa de pesquisa.

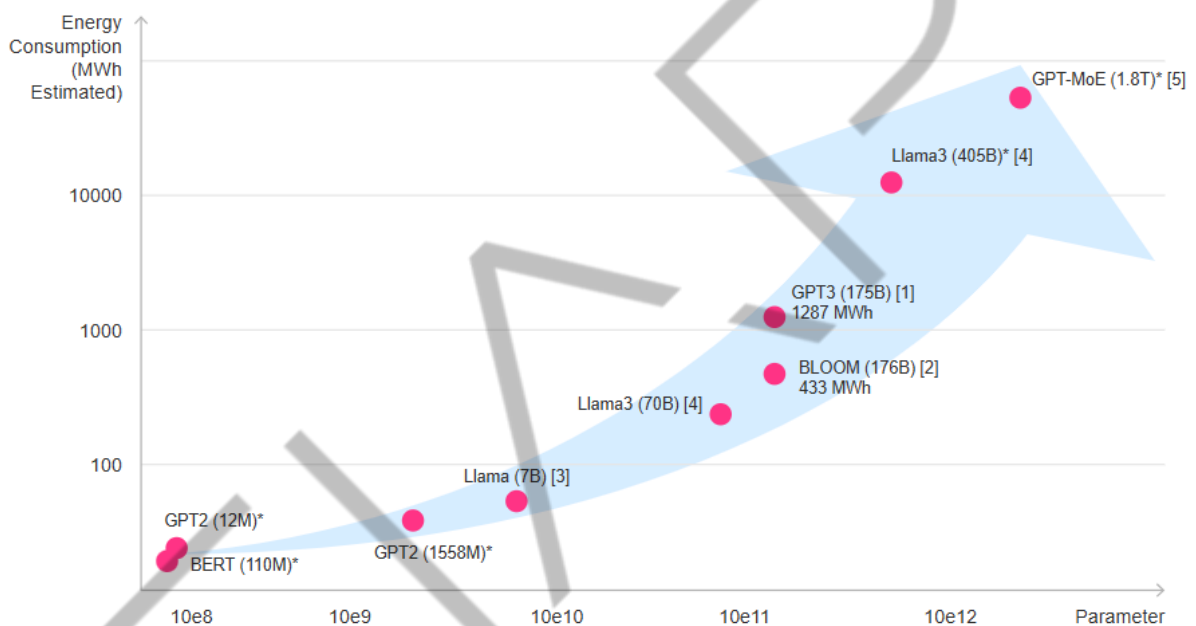


Figura 15 - Consumo de energia de modelos de IA Generativa
Fonte: ResearchGate (2024), adaptado por FIAP (2025)

Outra limitação crítica reside nos requisitos de dados. Modelos generativos de alto desempenho exigem vastas quantidades de dados de treinamento de alta qualidade para aprenderem efetivamente as complexas distribuições subjacentes. A qualidade, quantidade e curadoria desses dados são primordiais. Dados insuficientes ou de baixa qualidade podem resultar em modelos que geram conteúdo irrelevante, repetitivo ou com artefatos indesejados. Além disso, o processo de curadoria dos dados — coletar, limpar, filtrar e organizar — é trabalhoso e caro. Garantir que os dados de treinamento sejam representativos e minimizem vieses é um desafio ético e técnico, como discutido anteriormente. Em domínios especializados ou de nicho, onde grandes conjuntos de dados não estão prontamente disponíveis, o treinamento de um modelo generativo eficaz pode ser particularmente desafiador.

O controle e a direção da saída gerada também representam um desafio significativo. Embora técnicas como prompt engineering, ajuste fino (fine-tuning) e Aprendizado por Reforço a partir de Feedback Humano (RLHF) permitam algum grau de controle sobre o que o modelo gera, alcançar uma direção precisa e confiável pode ser difícil. Modelos generativos, por sua natureza, introduzem aleatoriedade e variabilidade. Fazer com que um modelo gere exatamente o que um usuário deseja, evitando saídas indesejadas ou inesperadas, requer prompts cuidadosos e, muitas vezes, iteração. Modelos de linguagem, por exemplo, podem ser sensíveis a pequenas mudanças na formulação do prompt. Modelos de imagem podem interpretar descrições de texto de maneiras inesperadas. Por isso, garantir que os modelos sigam instruções complexas ou restrições específicas de forma consistente é uma área de pesquisa em andamento.



Figura 16 - Alucinações e a dificuldade de se dar a instrução correta para a resposta desejada
Fonte: Unframed Digital (2024)

As questões de generalização e alucinação são preocupações sérias, especialmente em modelos de linguagem. Embora os modelos generativos sejam excelentes em gerar conteúdo que se parece com os dados de treinamento, eles podem ter dificuldade em generalizar para situações ou conceitos fora de sua distribuição de treinamento. Um aspecto ainda mais preocupante é o fenômeno da “alucinação”, onde o modelo gera informações que parecem factualmente corretas e

coerentes, mas na verdade são falsas ou sem base nos dados de treinamento. Isso representa um sério problema de veracidade e precisão, tornando essencial a verificação humana do conteúdo gerado por IA, especialmente em domínios críticos como notícias, saúde ou finanças. A falta de um verdadeiro “entendimento” ou raciocínio por parte do modelo significa que ele pode gerar conteúdo plausível, mas incorreto ou sem sentido no mundo real.

Por fim, a IA Generativa introduz novos riscos de segurança. A natureza dos modelos e suas aplicações os torna alvos potenciais para ataques maliciosos. A injeção de prompt ocorre quando um usuário malicioso manipula a entrada (prompt) de um modelo para fazê-lo gerar conteúdo prejudicial, ignorar salvaguardas ou revelar informações confidenciais. O sequestro de modelo (*model hacking*) pode envolver a exploração de vulnerabilidades para obter controle sobre o modelo ou seus dados subjacentes. O envenenamento de dados (*data poisoning*) é um ataque onde dados maliciosos são introduzidos no conjunto de treinamento para subverter o comportamento do modelo, fazendo-o gerar saídas tendenciosas ou prejudiciais. Garantir a segurança e a robustez dos modelos generativos contra essas e outras ameaças é um desafio crescente à medida que eles se tornam mais difundidos e poderosos.

Embora a IA Generativa tenha alcançado marcos notáveis, ela não está isenta de desafios. Os altos custos computacionais, a dependência de grandes conjuntos de dados de alta qualidade, as dificuldades em alcançar controle preciso sobre a geração, os problemas de veracidade e as vulnerabilidades de segurança são áreas ativas de pesquisa e desenvolvimento. Superar essas limitações será fundamental para liberar todo o potencial da IA Generativa de forma segura e equitativa.

2 LABORATÓRIO PYTHON: FUNDAMENTOS DE ÁLGEBRA LINEAR PARA IA

Este laboratório prático tem como objetivo introduzir os conceitos básicos de álgebra linear que são fundamentais para a compreensão de muitos algoritmos e modelos utilizados em Inteligência Artificial, especialmente nas áreas de Aprendizado de Máquina e Aprendizado Profundo. Utilizaremos a biblioteca NumPy, que é a ferramenta padrão em Python para computação numérica eficiente, lidando com vetores, matrizes e tensores.

A álgebra linear lida com estruturas como vetores e matrizes, bem como as operações que podem ser realizadas sobre elas. Em IA, dados de diversas naturezas, como imagens, texto, áudio ou conjuntos de características extraídas, são frequentemente representados numericamente na forma de vetores ou matrizes. As operações de álgebra linear constituem a base matemática para muitos cálculos essenciais, incluindo transformações de dados, processos de otimização (como o treinamento de modelos) e as operações internas das camadas em Redes Neurais.

Para começar, certifique-se de ter a biblioteca NumPy instalada em seu ambiente Python. Caso não a tenha, você pode instalá-la facilmente utilizando o gerenciador de pacotes pip: `pip install numpy`.

2.1 Vetores

Um vetor pode ser conceitualmente pensado como uma lista ordenada de números. Geometricamente, em espaços de duas ou três dimensões, um vetor representa uma direção e magnitude a partir da origem. Em contextos de IA, um vetor frequentemente representa as características (features) de um único ponto de dados. Por exemplo, se você está trabalhando com dados de casas e cada casa tem características como número de quartos, área e preço, um vetor pode conter os valores `[3, 150, 300000]` para uma casa específica. No NumPy, vetores são representados como arrays unidimensionais.

Vamos criar e realizar algumas operações básicas com vetores usando NumPy.

```
import numpy as np

--- Seção 1: Vetores ---
print("--- Seção 1: Vetores ---")

vetor_a = np.array([1, 2, 3])
vetor_b = np.array([4, 5, 6])

print(f"Vetor A: {vetor_a}")
print(f"Vetor B: {vetor_b}")
print(f"Dimensão do Vetor A: {vetor_a.shape}")

soma_vetores = vetor_a + vetor_b
print(f"Soma de A + B: {soma_vetores}")

subtracao_vetores = vetor_a - vetor_b
print(f"Subtração de A - B: {subtracao_vetores}")

escalar = 2
vetor_multiplicado = vetor_a * escalar
print(f"Vetor A multiplicado por {escalar}: {vetor_multiplicado}")

produto_escalar = np.dot(vetor_a, vetor_b)
print(f"Produto Escalar (Dot Product) de A e B: {produto_escalar}")

print("-" * 20)
```

Código-fonte 1 - Vetores no NumPy

Fonte: Elaborado pelo autor (2025)

Criamos dois vetores unidimensionais, `vetor_a` e `vetor_b`, usando a função `np.array()`. Exibimos os vetores e suas dimensões utilizando o atributo `.shape`, que, no caso de um vetor unidimensional, retorna o número de elementos. Realizamos operações de adição e subtração de vetores, que em NumPy são feitas elemento a elemento. Também demonstramos a multiplicação de um vetor por um escalar, onde cada elemento do vetor é multiplicado por esse valor. Finalmente, calculamos o Produto Escalar (Dot Product) dos dois vetores usando `np.dot()`. O produto escalar é uma operação fundamental que resulta em um único número e é amplamente utilizado em IA, como no cálculo da ativação de neurônios em Redes Neurais.

2.2 Matrizes

Uma matriz pode ser visualizada como uma grade retangular de números, organizada em linhas e colunas. Em IA, matrizes são estruturas de dados extremamente comuns. Um conjunto de dados tabular, por exemplo, pode ser representado como uma matriz onde cada linha corresponde a um ponto de dados (uma amostra) e cada coluna corresponde a uma característica. As matrizes também são utilizadas para representar pesos e transformações em Redes Neurais, assim como imagens (onde a matriz representa os valores de pixel). Em NumPy, as matrizes são arrays bidimensionais, criadas a partir de listas aninhadas, nas quais cada lista interna representa uma linha.

Vamos criar e realizar operações básicas com matrizes.

```
--- Seção 2: Matrizes ---
print("\n--- Seção 2: Matrizes ---")

matriz_x = np.array([[1, 2],
                     [3, 4]])

matriz_y = np.array([[5, 6],
                     [7, 8]])

print(f"Matriz X:\n{matriz_x}")
print(f"Matriz Y:\n{matriz_y}")
print(f"Dimensão da Matriz X: {matriz_x.shape}")

soma_matrizes = matriz_x + matriz_y
print(f"Soma de X + Y:\n{soma_matrizes}")

subtracao_matrizes = matriz_x - matriz_y
print(f"Subtração de X - Y:\n{subtracao_matrizes}")

escalar_matriz = 3
matriz_multiplicada = matriz_x * escalar_matriz
print(f"Matriz X multiplicada por {escalar_matriz}:\n{matriz_multiplicada}")

print("--- * 20)
```

Código-fonte 2 - Matrizes no NumPy
Fonte: Elaborado pelo autor (2025)

Neste código, criamos duas matrizes bidimensionais, `matriz_x` e `matriz_y`, utilizando `np.array()` com listas aninhadas. Exibimos as matrizes e suas dimensões usando o atributo `.shape`, que, no caso das matrizes, retorna uma tupla indicando o

número de linhas e colunas. Demonstramos a adição e subtração de matrizes, que, assim como nos vetores, são realizadas elemento a elemento, exigindo que as matrizes tenham as mesmas dimensões. Também mostramos a multiplicação de uma matriz por um escalar, no qual cada elemento da matriz é multiplicado pelo escalar.

2.2.1 Multiplicação de matrizes

Uma das operações mais importantes e frequentemente utilizadas em álgebra linear para IA é a multiplicação de matrizes. É crucial entender que a multiplicação de matrizes não é uma multiplicação elemento a elemento (essa operação é chamada de produto de Hadamard e é representada por $*$ em NumPy). A multiplicação de matrizes tem uma definição específica que é fundamental para o funcionamento de Redes Neurais, transformações geométricas e muitos outros algoritmos.

Para que duas matrizes, A e B , possam ser multiplicadas na ordem $A * B$, o número de colunas na matriz A deve ser igual ao número de linhas na matriz B . Se A tem dimensão $m \times n$ e B tem dimensão $n \times p$, o produto $C = A * B$ será uma matriz de dimensão $m \times p$. Cada elemento C_{ij} na matriz resultante é calculado como o produto escalar da i -ésima linha de A pela j -ésima coluna de B .

No NumPy, a multiplicação de matrizes pode ser realizada usando a função `np.dot()` ou o operador `@` (disponível a partir do Python 3.5).

```
print("\n--- Seção 3: Multiplicação de Matrizes ---")

matriz_a_mult = np.array([[1, 2, 3],
                           [4, 5, 6]]) # Matriz 2x3

matriz_b_mult = np.array([[7, 8],
                           [9, 10],
                           [11, 12]]) # Matriz 3x2

print(f"Matriz A (2x3):\n{matriz_a_mult}")
print(f"Matriz B (3x2):\n{matriz_b_mult}")

produto_matrizes = np.dot(matriz_a_mult, matriz_b_mult)
# Alternativamente, usando o operador @:
# produto_matrizes = matriz_a_mult @ matriz_b_mult

print(f"Produto de Matrizes A @ B (2x2):\n{produto_matrizes}")

# Tentando multiplicar matrizes com dimensões incompatíveis
# (isso geraria um erro)
# matriz_incompativel = np.array([[1, 2], [3, 4]]) # Matriz 2x2
# try:
#     np.dot(matriz_a_mult, matriz_incompativel)
# except ValueError as e:
#     print(f"\nTentando multiplicar matrizes incompatíveis (2x3
# e 2x2) resultaria em um erro: {e}")
```

Código-fonte 3 - Multiplicação de matrizes
Fonte: Elaborado pelo autor (2025)

Na Seção 3, criamos duas matrizes com dimensões compatíveis para multiplicação: `matriz_a_mult` (2x3) e `matriz_b_mult` (3x2). O número de colunas de A (3) é igual ao número de linhas de B (3), permitindo a multiplicação. O resultado é uma matriz de dimensão 2x2. Realizamos a multiplicação utilizando `np.dot()`. Além disso, é importante notar que a ordem da multiplicação de matrizes importa ($A @ B$ geralmente não é igual a $B @ A$), e tentar multiplicar matrizes com dimensões incompatíveis resultará em um erro `ValueError`.

2.2.2 Outras operações relevantes

O NumPy oferece uma vasta gama de outras operações de álgebra linear que são frequentemente úteis em IA. Vamos explorar brevemente algumas delas.

A inversa de uma matriz é um conceito fundamental em álgebra linear, análogo ao inverso de um número escalar (por exemplo, o inverso de 2 é $1/2$). Para uma matriz

quadrada A , sua inversa, denotada por A^{-1} , é a matriz que, quando multiplicada por A , resulta na matriz identidade: $A \times A^{-1} = A^{-1} \times A = I$, onde I é a matriz identidade de mesma dimensão que A . Nem todas as matrizes quadradas possuem uma inversa; matrizes que não possuem inversa são chamadas de singulares. A inversa de matrizes é crucial em tarefas como resolver sistemas de equações lineares, calcular mínimos quadrados e entender transformações lineares. No NumPy, você pode calcular a inversa de uma matriz utilizando a função `np.linalg.inv()`.

```
# Calculando a inversa de uma matriz
# A matriz deve ser quadrada e não singular
matriz_quadrada = np.array([[4, 1],
                             [3, 2]])

try:
    matriz_inversa = np.linalg.inv(matriz_quadrada)
    print(f"\nMatriz Quadrada:\n{matriz_quadrada}")
    print(f"Inversa da Matriz Quadrada:\n{matriz_inversa}")

    # Verificando a propriedade da inversa: A @ A_inv deve
    # ser a matriz identidade
    verificacao = np.dot(matriz_quadrada, matriz_inversa)
    print(f"Verificação (Matriz @ Inversa):\n{verificacao}")
    # Note que pode haver pequenos erros de ponto flutuante,
    # então os valores podem não ser exatamente 0s e 1s.
    # np.allclose(np.dot(matriz_quadrada, matriz_inversa),
    # np.eye(2)) pode ser usado para verificar se são "quase"
    # iguais.

except np.linalg.LinAlgError as e:
    print(f"\nNão foi possível calcular a inversa: {e}")
    print("A matriz pode ser singular (não invertível).")
```

Código-fonte 4 - Invertendo uma matriz
Fonte: Elaborado pelo autor (2025)

O determinante de uma matriz quadrada é um único número que fornece informações importantes sobre a matriz. Geometricamente, o valor absoluto do determinante de uma matriz 2x2 ou 3x3 representa a área ou volume do paralelogramo ou paralelepípedo formado pelos vetores coluna da matriz. Um determinante diferente de zero indica que a matriz é invertível (não singular), e que a transformação linear que ela representa não “achata” o espaço. O determinante é usado, por exemplo, no cálculo da inversa de uma matriz e na regra de Cramer para resolver sistemas lineares. Em NumPy, você pode calcular o determinante utilizando `np.linalg.det()`.

```
# Calculando o determinante de uma matriz
matriz_determinante = np.array([[4, 1],
                                [3, 2]])

determinante = np.linalg.det(matriz_determinante)
print(f"\nMatriz para Determinante:\n{matriz_determinante}")
print(f"Determinante da Matriz: {determinante}")

# Exemplo de matriz singular (determinante zero)
matriz_singular = np.array([[1, 2],
                             [2, 4]]) # A segunda linha é um
múltiplo da primeira

determinante_singular = np.linalg.det(matriz_singular)
print(f"\nMatriz Singular:\n{matriz_singular}")
print(f"Determinante da Matriz Singular:
{determinante_singular}") # Será próximo de zero devido a
erros de ponto flutuante
```

Código-fonte 5 - Determinante de uma matriz
Fonte: Elaborado pelo autor (2025)

As normas são medidas do “tamanho” ou “magnitude” de vetores e matrizes. Elas são utilizadas extensivamente em IA para medir distâncias, avaliar o tamanho de erros (como na função de custo), regularizar modelos para evitar o overfitting e analisar a convergência de algoritmos. A norma L2 (norma Euclidiana) é a mais comum para vetores, calculada como a raiz quadrada da soma dos quadrados dos elementos. Para matrizes, existem várias normas, como a norma de Frobenius ou normas induzidas. Em NumPy, `np.linalg.norm()` calcula diferentes normas.

```
# Calculando normas
vetor_norma = np.array([3, -4]) # Vetor 2D

# Norma L2 (Euclidiana) de um vetor: sqrt(3^2 + (-4)^2) =
sqrt(9 + 16) = sqrt(25) = 5
norma_l2_vetor = np.linalg.norm(vetor_norma)
print(f"\nVetor para Norma: {vetor_norma}")
print(f"Norma L2 do vetor: {norma_l2_vetor}")

# Norma de Frobenius de uma matriz: sqrt(soma dos quadrados
de todos os elementos)
matriz_norma = np.array([[1, 2],
                          [3, 4]])
norma_frobenius_matriz = np.linalg.norm(matriz_norma, 'fro')
print(f"\nMatriz para Norma:\n{matriz_norma}")
print(f"Norma de Frobenius da matriz:
{norma_frobenius_matriz}") # sqrt(1^2 + 2^2 + 3^2 + 4^2) =
sqrt(1+4+9+16) = sqrt(30) approx 5.
```

Código-fonte 6 - Norma de uma matriz
 Fonte: Elaborado pelo autor (2025)

As distâncias entre vetores são medidas de quão “longe” ou “diferentes” dois vetores estão um do outro. O conceito de distância é fundamental em muitos algoritmos de IA, como em algoritmos de agrupamento (clustering) onde agrupamos pontos de dados próximos, ou em algoritmos de classificação baseados em vizinhos mais próximos. A distância entre dois vetores $\mathbf{u}$ e $\mathbf{v}$ é definida como a norma de sua diferença: $Dist(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|$. As normas mais comuns usadas para calcular distâncias são a norma L2 (resultando na Distância Euclidiana) e a norma L1 (resultando na Distância de Manhattan). A Distância Euclidiana entre dois vetores é a distância em linha reta entre os pontos que eles representam no espaço. É calculada como a raiz quadrada da soma dos quadrados das diferenças entre seus elementos correspondentes. É simplesmente a norma L2 do vetor diferença: $DistEuclidean(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2$.

A Distância de Manhattan (ou distância da cidade em blocos, L1) é a soma dos valores absolutos das diferenças entre os elementos correspondentes dos vetores. É calculada como a norma L1 do vetor diferença: $DistManhattan(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_1 = \sum_i |\mathbf{u}_i - \mathbf{v}_i|$. Ela representa a distância que você percorreria entre dois pontos, considerando apenas movimentos horizontais e verticais (como em um grid de ruas de uma cidade).

No NumPy, podemos calcular essas distâncias usando `np.linalg.norm()` no vetor diferença, especificando o parâmetro `ord` para a norma desejada (2 para L2/Euclidiana, 1 para L1/Manhattan).

```
# Calculando distâncias entre vetores
vetor_u = np.array([1, 2])
vetor_v = np.array([4, 6])

# Vetor diferença
vetor_diferenca = vetor_u - vetor_v # [1-4, 2-6] = [-3, -4]
print(f"\nVetor U: {vetor_u}")
print(f"Vetor V: {vetor_v}")
print(f"Vetor Diferença (U - V): {vetor_diferenca}")

# Distância Euclidiana (Norma L2 da diferença)
distancia_euclidiana = np.linalg.norm(vetor_diferenca,
ord=2)
# Ou diretamente: np.linalg.norm(vetor_u - vetor_v, ord=2)
print(f"Distância Euclidiana entre U e V:
{distancia_euclidiana}") # sqrt((-3)^2 + (-4)^2) =
sqrt(9+16) = 5

# Distância de Manhattan (Norma L1 da diferença)
distancia_manhattan = np.linalg.norm(vetor_diferenca, ord=1)
# Ou diretamente: np.linalg.norm(vetor_u - vetor_v, ord=1)
print(f"Distância de Manhattan entre U e V:
{distancia_manhattan}") # |-3| + |-4| = 3 + 4 = 7
```

Código-fonte 7 - Distâncias entre vetores
Fonte: Elaborado pelo autor (2025)

A transposição de uma matriz é uma operação onde as linhas se tornam colunas e as colunas se tornam linhas. Se a matriz original tem dimensão $m \times n$, sua transposta terá dimensão $n \times m$. A transposição é indicada por um sobrescrito T (A^T). Em NumPy, você pode obter a transposta usando o atributo `.T` ou a função `np.transpose()`.

A matriz identidade é uma matriz quadrada (mesmo número de linhas e colunas) onde todos os elementos na diagonal principal são 1 e todos os outros elementos são 0. Ela é análoga ao número 1 na multiplicação escalar: multiplicar qualquer matriz pela matriz identidade (de dimensão compatível) resulta na matriz original. Em NumPy, `np.eye(n)` cria uma matriz identidade $n \times n$.

```
print("\n--- Seção 4: Outras Operações Úteis (Breve) ---")

matriz_original = np.array([[1, 2],
                             [3, 4],
                             [5, 6]]) # Matriz 3x2
matriz_transposta = matriz_original.T # Ou
np.transpose(matriz_original)
print(f"Matriz Original (3x2):\n{matriz_original}")
print(f"Matriz Transposta (2x3):\n{matriz_transposta}")

matriz_identidade_2x2 = np.eye(2)
print(f"Matriz Identidade 2x2:\n{matriz_identidade_2x2}")

# Demonstrando a propriedade da Matriz Identidade: Matriz X @
# Matriz Identidade = Matriz X
# Usaremos a Matriz X que criamos na Seção 2.
print(f"Matriz X:\n{matriz_x}")
print(f"Matriz X @ Matriz Identidade 2x2:\n{np.dot(matriz_x,
matriz_identidade_2x2)}")
```

Código-fonte 8 - Matriz transposta e matriz identidade

Fonte: Elaborado pelo autor (2025)

A função `np.linspace()` cria um array com um número especificado de amostras espaçadas uniformemente em um intervalo definido. Isso é útil para gerar sequências de valores para plotagem ou cálculos.

```
# Criando um array com valores linearmente espaçados
array_linspace = np.linspace(0, 10, 5) # 5 pontos entre 0 e
10 (inclusive)
print(f"\nArray criado com linspace (0 a 10, 5 pontos):
{array_linspace}")
```

Código-fonte 9 - Função Linspace

Fonte: Elaborado pelo autor (2025)

As funções `np.zeros()` e `np.ones()` são usadas para criar arrays de uma determinada forma e tipo, preenchidos com zeros ou uns, respectivamente. Isso é útil para inicializar matrizes, como pesos em Redes Neurais antes do treinamento.

```
# Criando um array preenchido com zeros
array_zeros = np.zeros((2, 3)) # Matriz 2x3 de zeros
print(f"\nArray de zeros (formato 2x3):\n{array_zeros}")

# Criando um array preenchido com uns
array_ones = np.ones((3, 2)) # Matriz 3x2 de uns
print(f"\nArray de uns (formato 3x2):\n{array_ones}")
```

Código-fonte 10 - Funções zeros e ones

Fonte: Elaborado pelo autor (2025)

A função `np.concatenate()` é usada para juntar uma sequência de arrays ao longo de um eixo existente. Isso é útil para combinar diferentes conjuntos de dados ou partes de matrizes.

```
# Concatenando arrays
array1 = np.array([[1, 2], [3, 4]])
array2 = np.array([[5, 6]])

# Concatenar ao longo do eixo 0 (linhas)
array_concatenado_linhas = np.concatenate((array1, array2),
axis=0)
print(f"\nArray 1:\n{array1}")
print(f"Array 2:\n{array2}")
print(f"Arrays concatenados ao longo do eixo 0
(linhas):\n{array_concatenado_linhas}")

# Para concatenar ao longo do eixo 1 (colunas), os arrays
devem ter o mesmo número de linhas
array3 = np.array([[7], [8]])
array_concatenado_colunas = np.concatenate((array1, array3),
axis=1)
print(f"\nArray 3:\n{array3}")
print(f"Arrays concatenados ao longo do eixo 1
(colunas):\n{array_concatenado_colunas}")
```

Código-fonte 11 - Concatenando matrizes e vetores
Fonte: Elaborado pelo autor (2025)

Outra operação comum é alterar a forma (`reshape`) de um array sem alterar seus dados. A função `np.reshape()` permite fazer isso, desde que o novo formato tenha o mesmo número total de elementos que o original.

```
# Alterando a forma (reshape) de um array
array_original_reshape = np.array([1, 2, 3, 4, 5, 6]) #
Vetor de 6 elementos
array_reshape = np.reshape(array_original_reshape, (2, 3)) #
Transformando em matriz 2x3
print(f"\nArray original
(vetor):\n{array_original_reshape}")
print(f"Array após reshape (matriz 2x3):\n{array_reshape}")
```

Código-fonte 12 - Função Reshape
Fonte: Elaborado pelo autor (2025)

Neste laboratório, foram abordadas operações básicas com vetores e matrizes utilizando a biblioteca NumPy, incluindo adição, subtração, multiplicação por escalar, produto escalar e multiplicação de matrizes. Também foram exploradas funções úteis para a criação de arrays (`linspace`, `zeros`, `ones`), concatenação, alteração de forma

(*reshape*), o cálculo da inversa e do determinante de uma matriz, bem como o cálculo de normas e distâncias entre vetores. Estes são os blocos de construção fundamentais para operações mais complexas em muitos algoritmos de IA Generativa. Como não é possível cobrir todas as funções de álgebra linear do NumPy, acesse a documentação da biblioteca para descobrir muitas outras funções úteis ou sempre que precisar tirar dúvidas quanto ao funcionamento de algum módulo.

Ao concluir este laboratório, você terá adquirido uma compreensão prática de como representar vetores e matrizes em Python usando NumPy e como realizar as operações de álgebra linear mais básicas que são onipresentes no campo da Inteligência Artificial. À medida que aprofundamos em tópicos como Redes Neurais Artificiais, Redes Neurais Recorrentes ou Transformers, verá que essas operações de álgebra linear são a base de seus cálculos internos. Praticar com NumPy é um excelente passo para construir uma base sólida para entender a matemática por trás da IA.

CONCLUSÃO

Neste capítulo, fizemos uma breve introdução ao conceito de IA Generativa e em que ponto nos encontramos nesse campo, ficando claro que estamos apenas no início de uma jornada transformadora. Vimos como essa tecnologia evoluiu de conceitos teóricos para sistemas capazes de criar conteúdo original (ou quase) e realista em diversas modalidades. Embora desafios significativos persistam, desde questões éticas e de segurança até limitações técnicas e computacionais, o ritmo da inovação sugere que as capacidades da IA Generativa continuarão a se expandir. À medida que avançamos, será essencial abordar esses desafios de forma proativa, garantindo que o desenvolvimento e a implantação da IA Generativa sejam guiados por princípios de responsabilidade, equidade e benefício para toda a sociedade, moldando um futuro onde a criatividade humana e a Inteligência Artificial possam coexistir e se aumentar mutuamente. Nos próximos capítulos, caminharemos no sentido de aplicar os principais conceitos e ferramentas para criar soluções complexas e começar a fazer parte deste mercado inovador.

REFERÊNCIAS

ABRAMSON, J.; ADLER, J.; DUNGER, J. *et al.* Accurate structure prediction of biomolecular interactions with AlphaFold 3. **Nature**, v. 630, p. 493–500, 08 de maio de 2024. Disponível em: <<https://doi.org/10.1038/s41586-024-07487-w>>. Acesso em: 13 maio 2025.

AI coding assistant refuses to write code, advises users to learn programming. **Gigazine**, 14 de março de 2025. Disponível em: <https://gigazine.net/gsc_news/en/20250314-ai-coding-assistant-refuses-learn-programming>. Acesso em: 13 maio de 2025.

BANSAL, Harsh. Text Generation Using LSTM. **Medium**, 17 de julho de 2020. Disponível em: <<https://bansalh944.medium.com/text-generation-using-lstm-b6ced8629b03>>. Acesso em: 13 maio 2025.

BRZICA, Darko. Google Core Algorithm Update – March 2024 – What It Really Means for the Future of SEO. **Unframed Digital**, 28 de março de 2024. Disponível em: <<https://www.unframeddigital.com/google-core-update-march-2024/>>. Acesso em: 13 maio 2025.

CRISTOFARO, de Emiliano. Na Overview of Privacy in Machine Learning. **ResearchGate**, maio 2020. Disponível em: <https://www.researchgate.net/publication/341478640_An_Overview_of_Privacy_in_Machine_Learning>. Acesso em: 13 maio 2025.

FOONG, Ng Wai. Introduction to VideoFusion. **Medium**, 30 de março de 2023. Disponível em: <<https://ngwaifoong92.medium.com/introduction-to-videofusion-58f8948b487a>>. Acesso em: 9 junho 2025.

GOODFELLOW, Ian J. *et al.* Generative adversarial nets. **Advances in neural information processing systems**, v. 27, 2014.

HOCHREITER, S.; SCHMIDHUBER, J., Long Short-Term Memory. **Neural Computation**, v. 9, n. 8, p. 1735-1780, 15 de novembro de 1997. Disponível em: <<https://doi.org/10.1162/neco.1997.9.8.1735>>. Acesso em: 13 maio 2025.

INTRODUCING AudioCraft: A Generative AI Tool For Audio and Music. **Meta**, 02 de Agosto de 2023. Disponível: <<https://about.fb.com/news/2023/08/audiocraft-generative-ai-for-music-and-audio/>>. Acesso em: 22 maio 2025.

JUMPER, John; HASSABIS, Demis. Protein structure predictions to atomic accuracy with AlphaFold. **Nature Methods**, [s. l.], v. 19, p. 11-12, 11 jan. 2022. Disponível em: <<https://www.nature.com/articles/s41592-021-01362-6#citeas>>. Acesso em: 13 maio 2025.

KINGMA, Diederik P.; WELLING, Max. **Auto-encoding variational bayes**. 20 de dezembro de 2013.

KLEINA, N. C. M. ChatGPT adia criador de imagens com IA porque usuários estavam abusando. **TecMundo**, 27 de março de 2025. Disponível em: <<https://www.tecmundo.com.br/internet/403610-chatgpt-adia-criador-de-imagens-com-ia-porque-usuarios-estavam-abusando.htm>>. Acesso em: 13 maio 2025.

LIXIA_SONDAR. O que aprendi em 18 meses de design de UI conversacional com IA. **r/UXDesign, Reddit**, abril de 2025. Disponível em: <https://www.reddit.com/r/UXDesign/comments/1ju90qt/what_ive_learned_from_18_mths_of_ai/?tl=pt-br>. Acesso em: 13 maio 2025.

LI, Yuzhuo, *et al.* The Unseen AI Disruptions for Power Grids: LLM-Induced Transients. **ResearchGate**, setembro de 2024. Disponível em: <https://www.researchgate.net/publication/384115745_The_Unseen_AI_Disruptions_for_Power_Grids_LLM-Induced_Transients>. Acesso em: 13 maio 2025.

LOBL, Edoardo. Além das Sequências Curtas: Como a IA está Aprendendo a Lidar com Contextos Extensos. **Dio**, 07 de maio de 2024. Disponível em: <<https://www.dio.me/articles/alem-das-sequencias-curtas-como-a-ia-esta-aprendendo-a-lidar-com-contextos-extensos>>. Acesso em: 22 maio 2025.

MACHINE Learning X Deep Learning. **IA Responsável**, 07 de setembro de 2023. Disponível em: <<https://www.iaresponsavel.com.br/2023/09/07/machine-learning-x-deep-learning/>>. Acesso em: 22 maio 2025.

PEREIRA, Fabio. Engenharia de Prompt – História dos modelos de linguagem – Parte 3. **Medium**, 23 de janeiro de 2024. Disponível em: <<https://medium.com/@fabio.alvaro/engenharia-de-prompt-hist%C3%B3ria-dos-modelos-de-linguagem-parte-3-5d7e662a3459>>. Acesso em: 13 maio 2025.

PRAMOD, Om. Hidden Markov Models: The Secret Sauce in Natural Language Processing. **Medium**, 30 de dezembro de 2023. Disponível em: <<https://medium.com/@ompramod9921/hidden-markov-models-the-secret-sauce-in-natural-language-processing-048e2baaa584>>. Acesso em: 13 maio 2025.

ROMBACH, Robin, *et al.* **High-resolution image synthesis with latent diffusion models**. Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2022.

SARASWAT, Siddharth. GANs: Simply Explained! **Medium**, 8 de novembro de 2023. Disponível em: <<https://medium.com/@siddharthsaraswat1/gans-simply-explained-0fd3dfc8593b>>. Acesso em: 13 maio 2025.

VASWANI, Ashish, *et al.* Attention Is All You Need. **Advances in neural information processing systems**, v. 30, 2017.

WEIZENBAUM, Joseph. ELIZA — a computer program for the study of natural language communication between man and machine. **Communications of the ACM**, v. 9, p. 36–45, 1966.